

Computer Architecture

Course No. 21472

Week #2

Performance

Bumsoo Kim

School of Computer Science & Engineering

E-mail : bumsoo@cau.ac.kr

Office : 310-430

About this Course

How does CPU work?

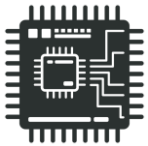
Source code

```
#include <stdio.h>

int main()
{
    printf("hello, world!");
    return 0;
}
```



CPU





Output

```
Hello World.
```

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

About this Course

How does CPU work?

Source code

```
#include <stdio.h>

int main()
{
    printf("hello, world!");
    return 0;
}
```

- Programming
  - Programming Languages
  - Data Structure
  - Algorithms

CPU

Output

```
Hello World.
```

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

About this Course

How does CPU work?

Source code

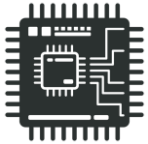
```
#include <stdio.h>

int main()
{
   printf("hello, world!");

   return 0;
}
```



CPU

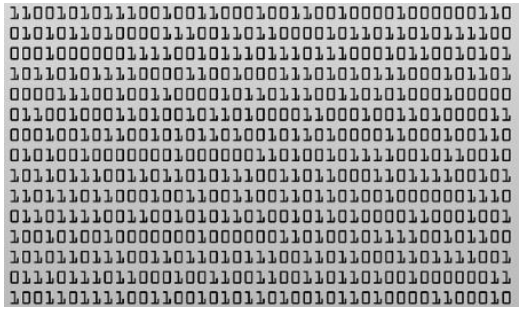


Output

```
Hello World.
```

Compiler

- Translates programming language into machine-friendly language
- ASM (Assembly Language)
- Machine Language



```
*************************************
* FUNCTION: INITA - Initialize ACIA
* INPUT: none
* OUTPUT: none
* CALLS: none
* DESTROYS: acc A

RESETA  EQU     %00010011
CTLREG  EQU     %00010001

INITA   LDA A   #RESETA    RESET ACIA
        STA A   ACIA
        LDA A   #CTLREG    SET 8 BITS AND 2 STOP
        STA A   ACIA

        JMP     SIGNON     GO TO START OF MONITOR
```

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

About this Course

How does CPU work?

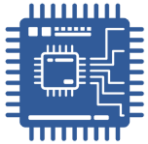
Source code

```
#include <stdio.h>

int main()
{
    printf("hello, world!");
    return 0;
}
```



CPU





Output

```
Hello World.
```

- Performance
- ISA (Instruction Set Architecture)
- Arithmetic for Computers
- Processor
- Pipelining
- Memory

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

About this Course

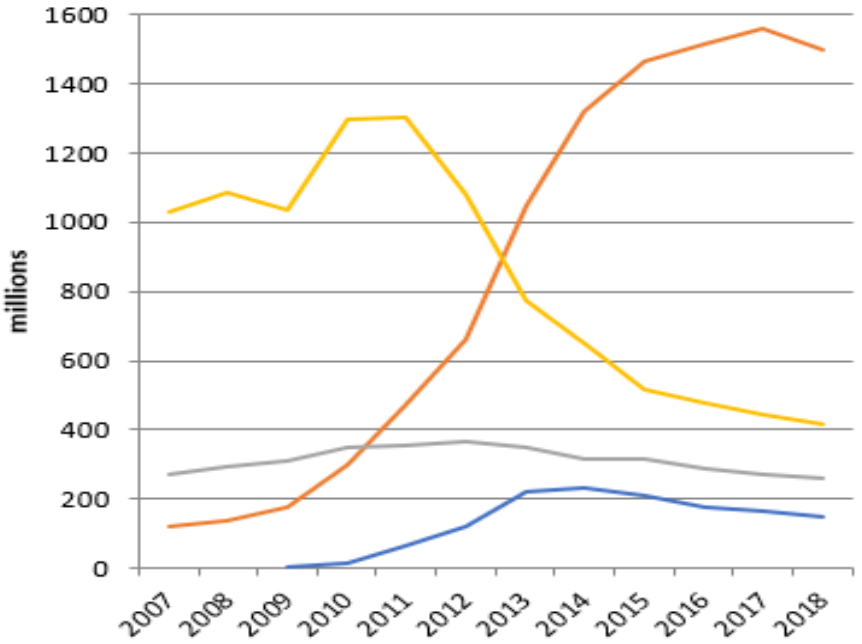
In this course, we learn:

What is a CPU and how does it work?

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

The Computer Revolution

The Post-PC Era



Smart phone

Cell phone (excluding smart phones)

PC (excluding tablets)

Tablet



Personal Mobile Devices (PMD)

- Computers are everywhere now!
- Battery operated

Cloud Computing

- **S**oftware **a**s **a** **S**ervice (SaaS)
- Enables software to run on PMD + Cloud

https://www.stratosphere networks.com/blog/turning-mobile-devices-into-business-tools/

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Classes of Computers



Supercomputers

- Purpose: High-end scientific & engineering calculations
- Performance measure : FLOPS > MIPS
- 10^{17} (desktop 10^{11}~10^{13}, about x10,000 faster)

FLOPS : **FL**oating-point **O**perations **P**er **S**econd

MIPS : **M**illion **I**nstructions **P**er **S**econd



Server Computers

- Purpose: Network based high-performance processing
- High capacity, performance, reliability
- Capacity and size varies



Personal Computers

- Purpose: General purpose, variety of software
- Subject to cost/performance trade-off

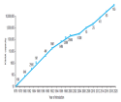


Embedded Computers

- Purpose: Hidden as components of systems
- Stringent power/performance/cost constraints

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Eight Great Ideas in Computer Architecture



Design for *Moore's Law*



Use *abstraction* to simplify design



Make the *common case fast*



Hierarchy of memories



Dependability via redundancy



Performance *via parallelism*



Performance *via pipelining*



Performance *via prediction*

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Below Your Program

Application Software (App)



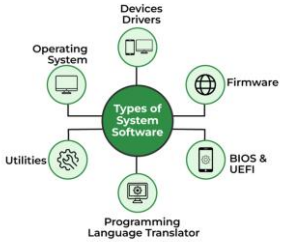
- Web browsers (Google Chrome, Firefox, etc.)
- Smartphone apps (Instagram, Kakaotalk, Candy Crush etc.)
- Microsoft Word, Excel, Powerpoint, etc.
- Written in high-level language



System Software

* GCC : **G**NU **C**ompiler **C**ollection

* GNU : Unix-like Operation System (**G**NU's **N**ot **U**nix)



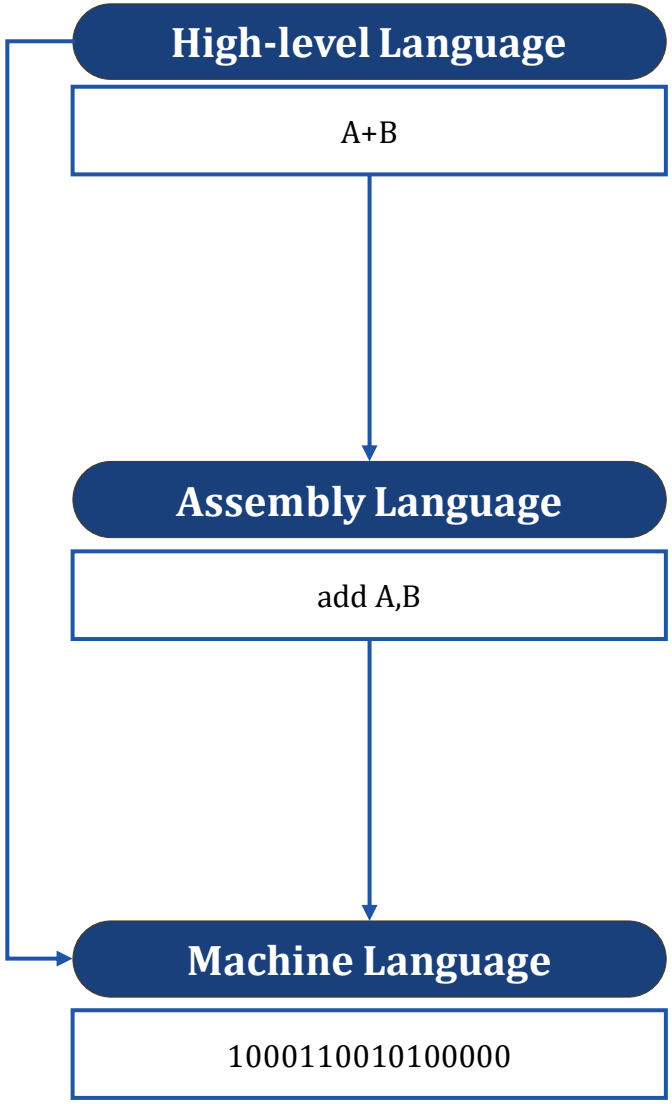
- Compiler: Translates HLL code to machine code (e.g., GCC)
- Operating System: Service code (e.g., GNU)
  - Handling the input/output
  - Managing memory and storage
  - Scheduling tasks & sharing resources

Hardware

- Processor, Memory, I/O controllers

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Compiler: Levels of Program Code



High-level language program (in C)

```
swap(int v[], int k)
{int temp;
   temp = v[k];
   v[k] = v[k+1];
   v[k+1] = temp;
}
```

Compiler

Assembly language program (for MIPS)

```
swap:
      muli $2, $5,4
      add  $2, $4,$2
      lw   $15, 0($2)
      lw   $16, 4($2)
      sw   $16, 0($2)
      sw   $15, 4($2)
      jr   $31
```

Assembler

Binary machine language program (for MIPS)

```
00000000101000010000000000011000
00000000000011000000110000100001
10001100011000100000000000000000
10001100111100100000000000000100
10101100111100100000000000000000
10101100011000100000000000000100
00000011111000000000000000001000
```

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

OS: How Apps Interact with OS

Application Initialization & Termination

- Loading: Loads executable file (libraries, dependency) into memory
- Process Creation: Creates process – allocating memory for code & data

System Calls

- Request services from OS (e.g., file operations, memory management, device handling, etc.)

Resource Management

- Memory management: allocates (virtual) memory for applications
- CPU scheduling: Schedules CPU time for application (when and how long)
- I/O management: Managing data transfer between application and external devices

Inter-Process Communication (IPC)

- Provides mechanisms for communicating with other processes
  (e.g., pipes, message queues, shared memory, or sockets)

Event handling

- Manages events like user inputs, hardware interrupts and notifies application.
- Apps typically register event handlers to respond to these events.

Error handling

- In error cases, enforces security policies to protect system resources and application integrity.

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Components of Computer

Processor

- Datapath: Actual Data Processing (arithmetic, logical, and memory access operations)
- Control Unit: Directs operation of datapath by interpreting the instructions fetched from memory and generate control signals

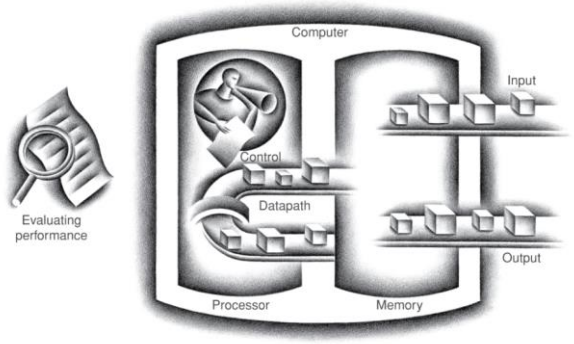
Datapath components

ALU : **A**rithmetic **L**ogic **U**nit (e.g., ADD, SUB, AND, OR)

Registers : Small, fast storage units that temporarily hold data/instructions

MUX : Multiplexers, control flow of data between different units

Buses : Pathways for data to travel between different parts of processor



* LAN : **L**ocal **A**rea **N**etwork

Input/Output

- User-interfaces devices: Display, keyboard, mouses, etc.
- Storage devices: Hard disk, CD/DVD, Flash, etc.
- Network adapters: LAN, WAN, Wireless (Wi-Fi, Bluetooth)

Communicate via **buses**
(Address Bus, Data Bus, Control Bus)
and memory controllers according to
memory hierarchy

* RAM : **R**andom **A**ccess **M**emory, * ROM : **R**ead-**O**nly **M**emory

Memory

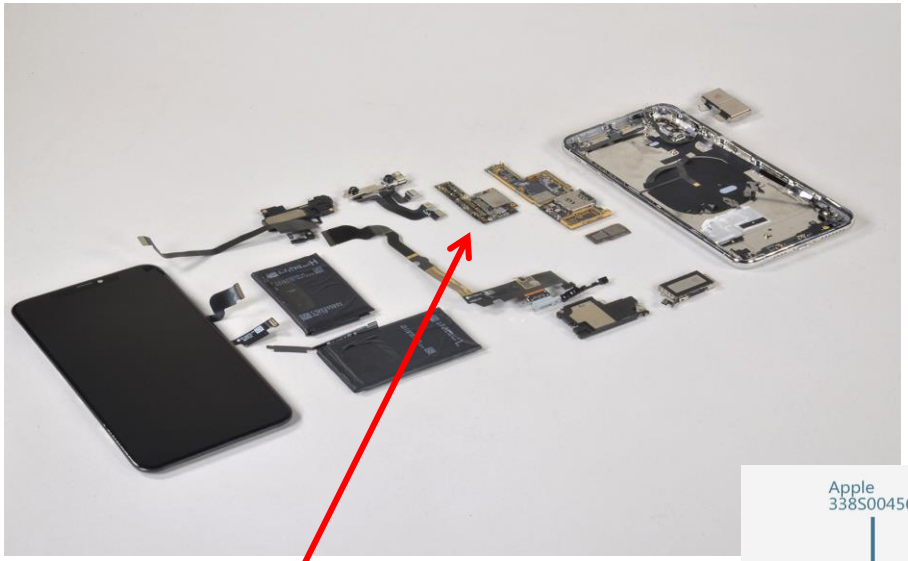
RAM : Volatile memory that stores data/instructions CPU needs to access quickly. Lost when off.

ROM : Non-volatile, Permanent instructions to start up computer (BIOS/firmware)

- Primary(Main) Memory: RAM (DRAM, SRAM)
- Secondary Memory: HDD, SSD
- Cache Memory: Small, high-speed memory closer to CPU than RAM
  (usually SRAM are used for cache)

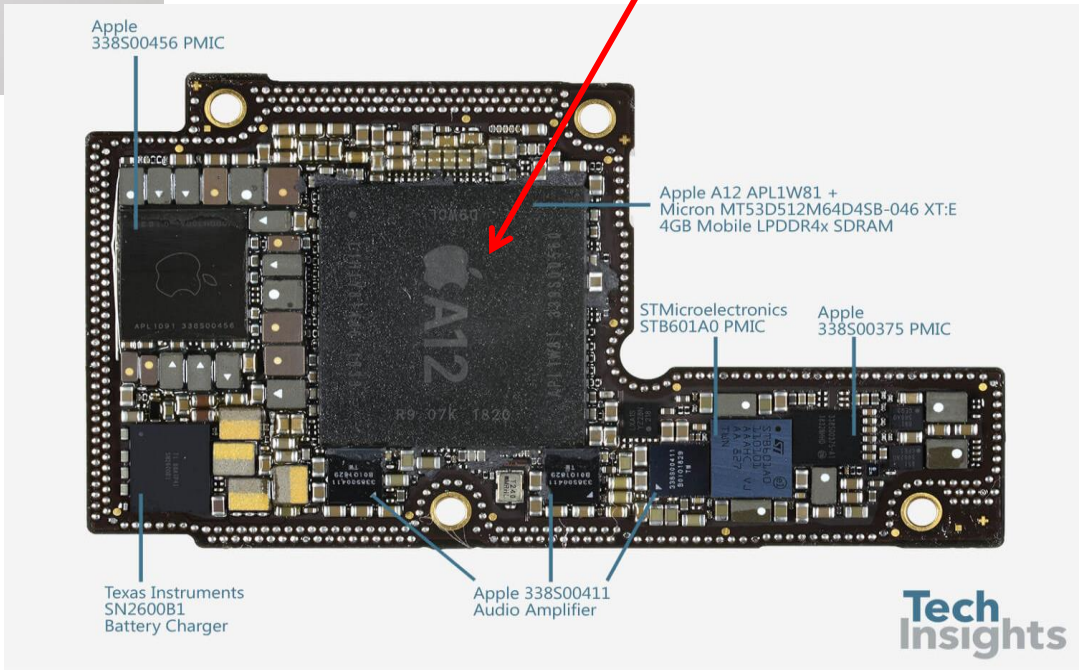
Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Opening the Box



Computer board

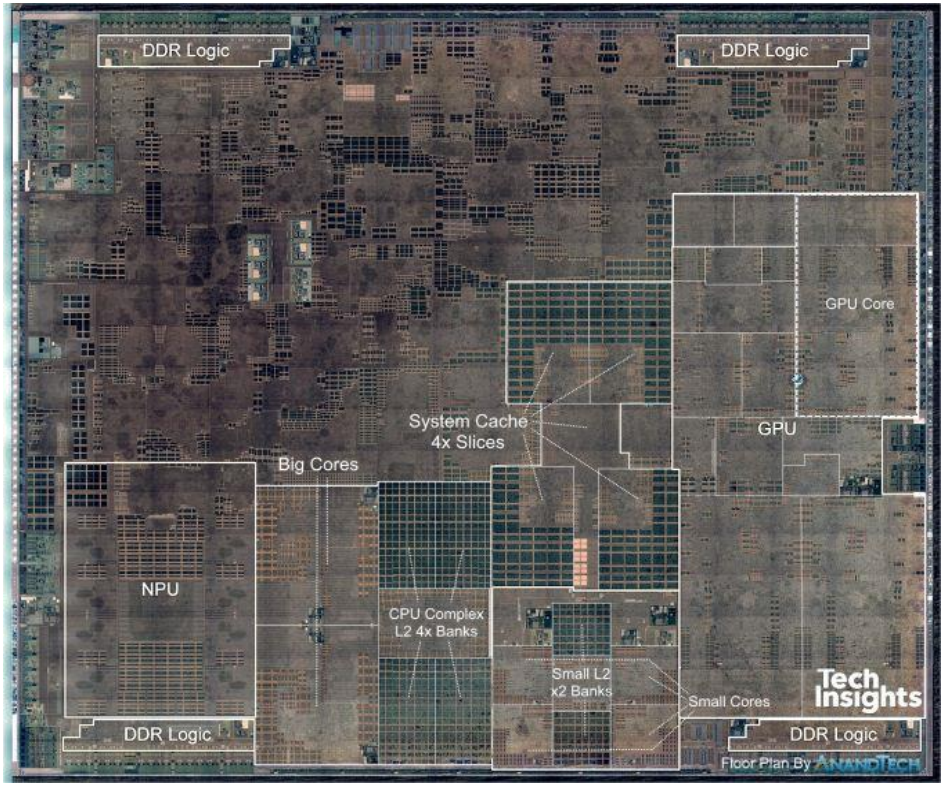
Processor



Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Inside the Processor (CPU)

Example : A12 processor

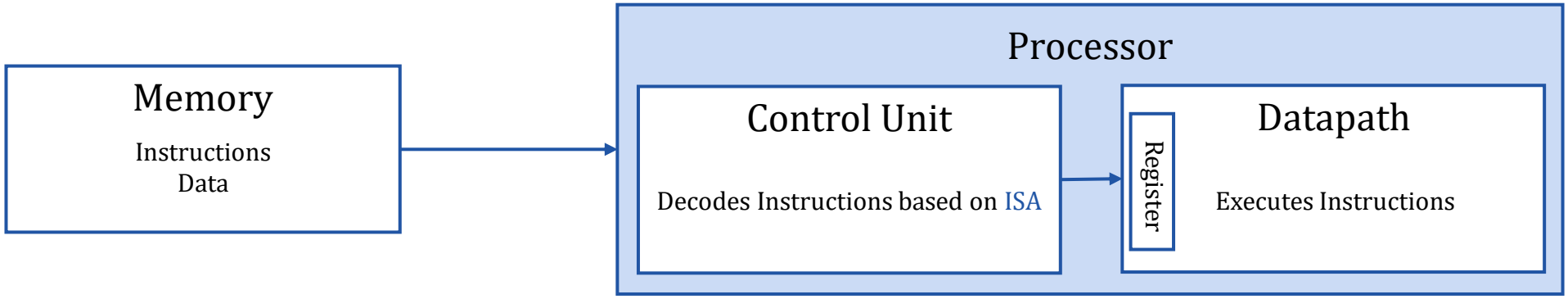


DDR (Double Data Rate)

- Memory technology for RAM. (e.g., DDR5)
- Designed to improve data transfer speeds by allowing data to be transfer at both rising and falling edge of the clock signal.

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Abstractions and ISA



Register : Fastest memory in computer system located directly within the processor.

Abstraction helps us deal with complexity

Hide lower-level detail

Instruction Set Architecture (ISA)

- Interface between hardware and low-level software
- Very important abstraction (How a processor interprets and executes instructions).
- Allowing developers to write software without needing to understand detailed inner workings of the hardware.
- Portability: Hardware can evolve without requiring changes in the software.
- Contract of how instructions are executed, what registers are available, how data is accessed and manipulated.

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Understanding Performance

Algorithm

Determines the number of operations executed

e.g., for the same operation "sorting", Bubble sort has $\frac{n^2}{2}$ operations while Merge sort has $n \log n$ operations.

Programming language, compiler, architecture

Determines the number of machine instructions executed per operation

- **Code in Assembly (x86 architecture):**

```assembly
mov eax, [a]    ; Load the value of 'a' into register eax
add eax, [b]    ; Add the value of 'b' to register eax
mov [x], eax    ; Store the result back into 'x'
```

- **Generated bytecode (Python bytecode example):**

```python
LOAD_FAST 0 (a)
LOAD_FAST 1 (b)
BINARY_ADD
STORE_FAST 2 (x)
```

  - **Number of machine instructions:**
    - 3 machine instructions: `mov`, `add`, `mov`

Processor and Memory system

Determine how fast instructions are executed

I/O System (including OS)

Determines how fast I/O operations are executed

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

https://www.stratospherenetworks.com/blog/turning-mobile-devices-into-business-tools/

Moore's Law and Performance

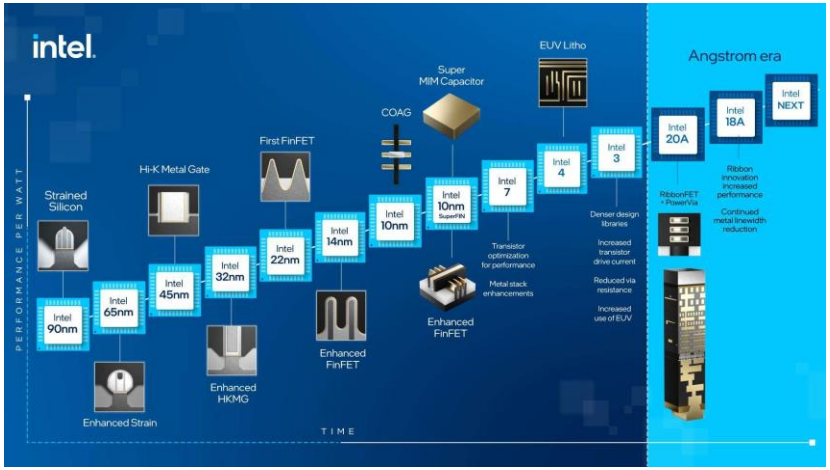
- Electronic technology continues to evolve

  Increased capacity and performance

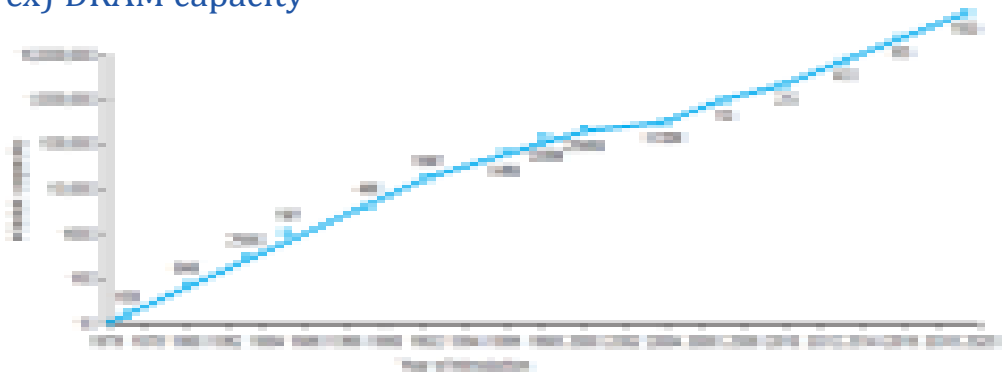
  Reduced cost

- Moore's Law

  IC capacity doubles every 18-24 months



ex) DRAM capacity

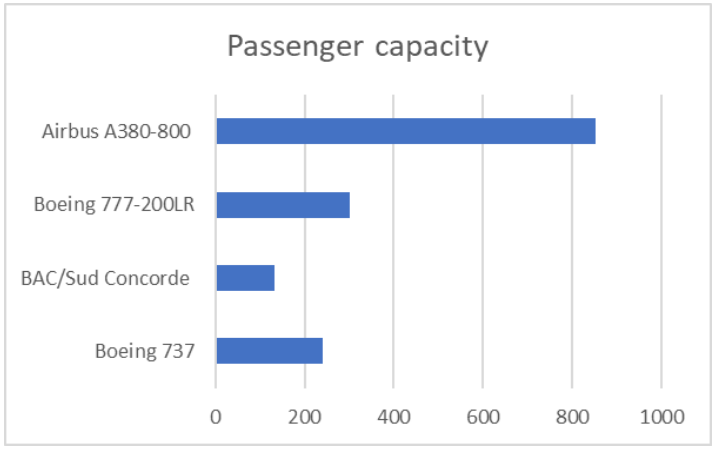


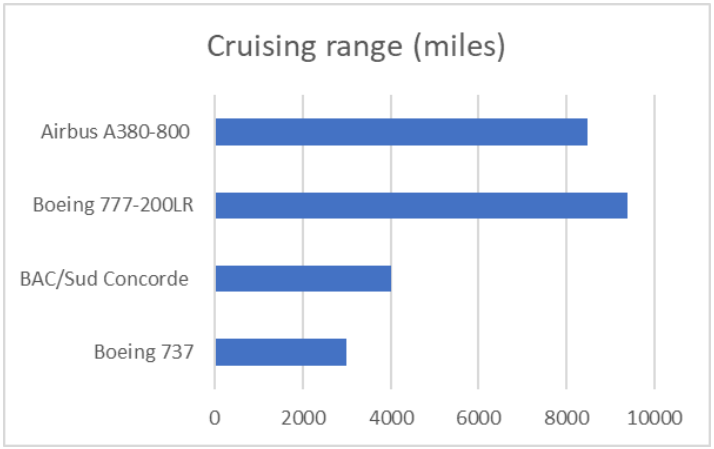
Year	Technology	Relative performance/cost
1951	Vacuum tube	1
1965	Transistor	35
1975	Integrated circuit (IC)	900
1995	Very large scale IC (VLSI)	2,400,000
2013	Ultra large scale IC	250,000,000,000

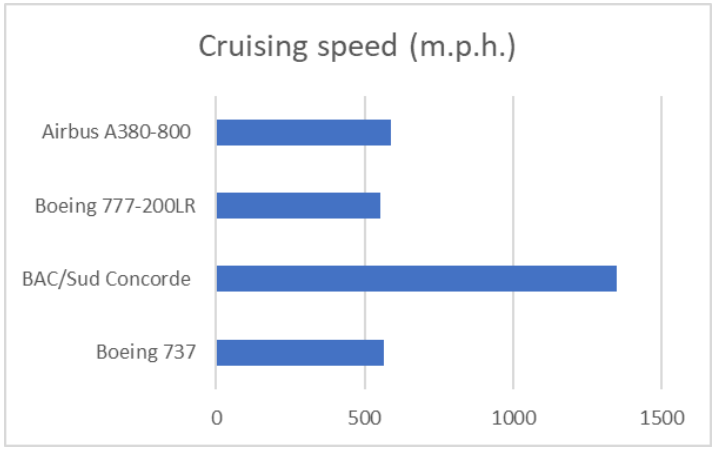
Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

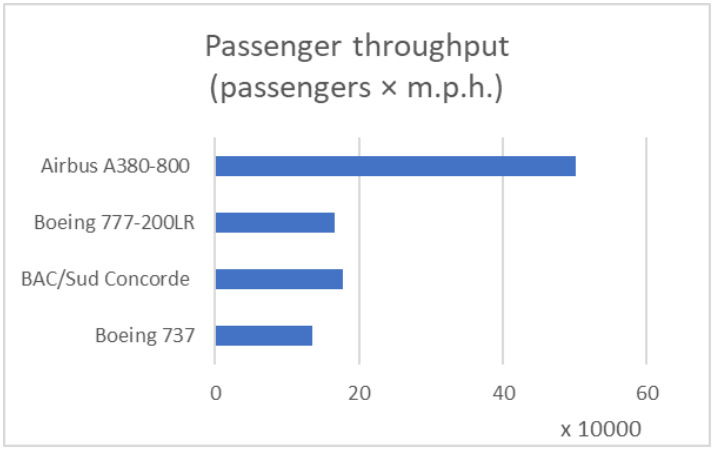
Defining Performance

Q. Which airplane has the best performance?









Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Response Time & Throughput

- Response time (latency, execution time)

  How long does it take to do a task/job?

- Throughput

  How many jobs can the machine run at once?

  Total work done per unit time (e.g., tasks / transcations / …per hour)

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Response Time & Throughput

- Response time (latency, execution time)

  How long does it take to do a task/job?

- Throughput

  How many jobs can the machine run at once?

  Total work done per unit time (e.g., tasks / transcations / …per hour)

Let's focus on Response time for now…

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Comparing Performance

- Response time (latency, execution time)
  - How long does it take to do a task/job?

$$\text{Performance}_X = \frac{1}{\text{Execution time}_X}$$

  - X is n time faster than Y

$$\frac{\text{Performance}_X}{\text{Performance}_Y} = \frac{\text{Execution time}_Y}{\text{Execution time}_X} = n$$

Example

There are two systems A and B. For an exactly identical program, it takes 7s on A and 21s on B.

How faster is A compared to B?

A.

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Measuring Execution Time

Elapsed time

- Total response time, including all aspects
- Determines system performance
- Useful, but often not good for comparison purposes

CPU time

- Time spent by CPU processing a given job
- CPU time = user CPU time + system CPU time
- User CPU time = User mode, time running actual instructions of the program
- System CPU time = Kernel model, hardware access, I/O, memory management

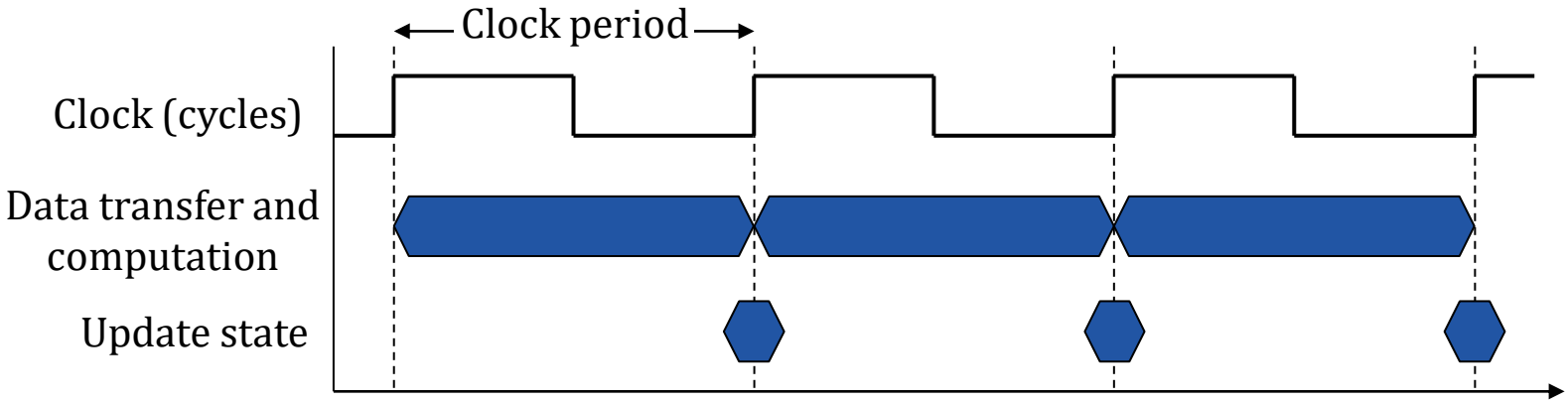
We focus on 'user CPU time'

- Time spent executing the lines of code that are in our program
- Further reference of Execution time = User CPU time
- But sometimes, other factors do matter for performance

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

CPU Clocking

- Operation of digital hardware governed by a constant-rate "clock".



- Clock frequency (Clock rate)
  - Cycles per second
  - e.g., 4.0GHz = 4000MHz = 4.0×10^9Hz
- Clock period
  - Duration of clock cycle
  - e.g., 250ps = 0.25ns = 250×10^{-12}s

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

CPU Time

- Performance is determined by execution time (=user CPU time)

$$\text{CPU Time} = (\text{CPU Clock Cycles}) \times (\text{Clock Cycle Time})$$

$$= \frac{\text{CPU Clock Cycles}}{\text{Clock Rate}}$$

- Performance improved by
  - Reducing number of clock cycles
  - Increasing clock frequency (or clock rate)
  - Trade-off often exists between clock rate against cycle count (clock cycles)

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

CPU Time

Example

Computer A: Clock frequency 2GHz, 10s CPU time

We are designing Computer B that aims for 6s CPU time. We can accelerate CPU clock at the cost of × 1.2 clock cycles than A.

How fast must the Clock Rate of Computer B should be?

A.

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Instruction Count, CPI, and Performance

- Instruction Count for a program
  - Determined by program, ISA and compiler

- CPI (clock Cycles Per Instruction)
  - Determined by CPU hardware
  - If different instructions have different CPI, calculate Average CPI for instruction mix

- Then, the performance (=Execution time=CPU Time) is calculated as

$$\text{CPU Time} = (\text{CPU Clock Cycles}) \times (\text{Clock Cycle Time})$$

$$\text{Clock Cycles} = (\text{Instruction Count}) \times (\text{Cycles Per Instruction})$$

$$\text{CPU Time} = (\text{Instruction Count}) \times \text{CPI} \times (\text{Clock Cycle Time})$$

$$= \frac{\text{Instruction Count} \times \text{CPI}}{\text{Clock Rate}}$$

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Instruction Count, CPI, and Performance

Example

Computer A: Cycle time is 250ps, and CPI 2.0

Computer B: Cycle time is 500ps, and CPI 1.2

Which Computer is faster, and by how much?

A.

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

CPI in More Detail

- If different instruction classes take different numbers of cycles

$$\text{Clock Cycles} = \sum_{i=1}^{n} (CPI_i \times \text{Instruction Count}_i)$$

- Weighted Average CPI

$$CPI = \frac{\text{Clock Cycles}}{\text{Instruction Count}} = \sum_{i=1}^{n} \left(CPI_i \times \underbrace{\frac{\text{Instruction Count}_i}{\text{Instruction Count}}}_{\text{Relative frequency}} \right)$$

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

CPI in More Detail

Example

Alternative compiled Code sequences using instructions in classes A, B, C

(for same computer, same high-level language code)

Class	A	B	C
CPI for class	1	2	3
IC in sequence 1	2	1	2
IC in sequence 2	4	1	1

Calculate Average CPI for Sequence 1 and 2 each

A.

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Performance Summary

- The final CPU Performance (=CPU Time) is written as:

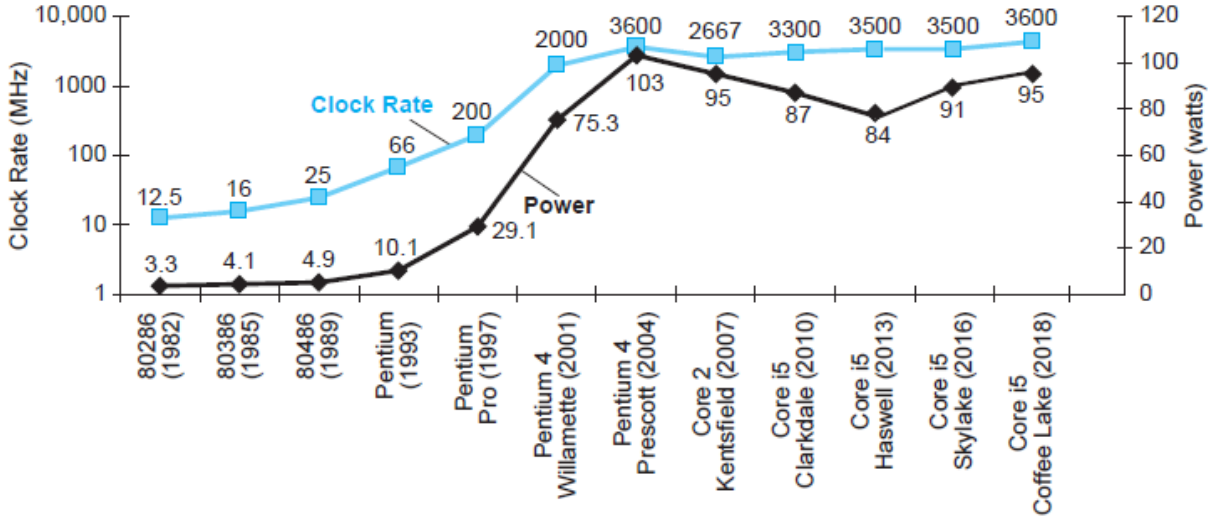
$$\text{CPU Time} = \left(\frac{\text{Instructions}}{\text{Program}}\right) \times \left(\frac{\text{Clock cycles}}{\text{Instruction}}\right) \times \left(\frac{\text{Seconds}}{\text{Clock cycle}}\right)$$

- Performance Depends on

	instruction count	CPI	clock rate
Algorithm/Program	✔	✔	
Programming language	✔	✔	
Compiler	✔	✔	
ISA	✔	✔	
Hardware organization		✔	✔
Technology			✔

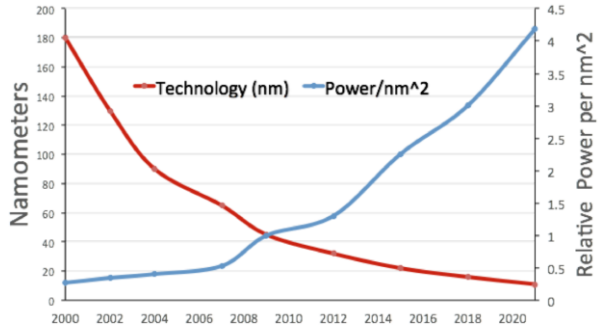
Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Power Trends



- In CMOS IC technology

$$\text{Power} \propto \frac{1}{2} \times \text{Capacitive load} \times \text{Voltage}^2 \times \text{Frequency}$$



Energy scaling for fixed task is better, since more and faster transistors

Power consumption based on models in "Dark Silicon and the End of Multicore Scaling," Hadi Esmaelizadeh, *ISCA*, 2011

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Reducing Power

- Suppose a new CPU has
  - 85% capacity load of old CPU
  - 15% voltage and 15% frequency reduction

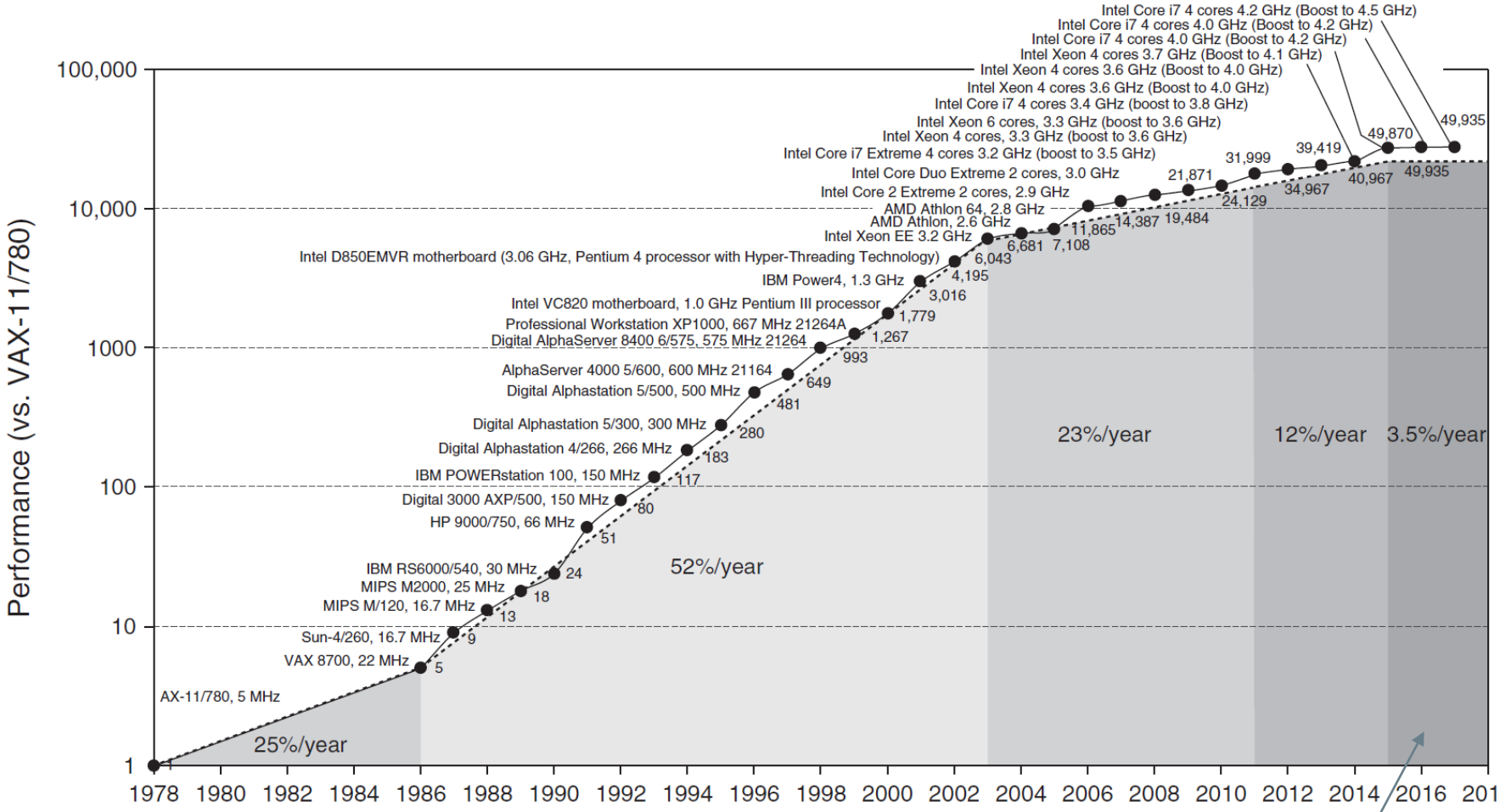
$$\frac{P_{new}}{P_{old}} = \frac{C_{old} \times 0.85 \times (V_{old} \times 0.85)^2 \times F_{old} \times 0.85}{C_{old} \times V_{old}{}^2 \times F_{old}} = 0.85^4 = 0.52$$

- The power wall
  - We can't reduce voltage further
  - We can't remove more heat

- How else can we improve performance?

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Uniprocessor Performance



Constrained by power, instruction-level parallelism, memory latency

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Multiprocessors

- Multicore microprocessors
  - More than one processor per chip
- Requires explicitly parallel programming
  - Compare with instruction level parallelism
    - Hardware executes multiple instructions at once
    - Hidden from the programmer
  - Hard to do
    - Programming for performance
    - Load balancing
    - Optimizing communication and synchronization

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

SPEC Benchmark

- Standard Performance Evaluation Corp
  - Develops benchmarks for CPU, I/O, Web, …
- Common set of programs used to measure performance
  - Supposedly typical of actual workload
- SPEC CPU benchmark
  - Ex: SPEC CPU 2006
  - Elapsed time to execute a selection of programs
    - Negligible I/O, so focuses on CPU performance
- SPEC Power benchmark
  - Power consumption of server at different workload levels

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Pitfall: Amdahl's Law

- Improving an aspect of a computer and expecting a proportional improvement in overall performance

$$T_{improved} = \frac{T_{affected}}{(improvement\ factor)} + T_{unaffected}$$

Example

Out of 100s program, multiply accounts for 80s. To achieve 5x performance overall, how much improvement in multiplication performance should be made?

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Fallacy: Low Power at Idle

- Look back at i7 power benchmark
  - At 100% load : 258W
  - At 50% load : 170W (66%)
  - At 10% load : 121W (47%)

Target Load %	Performance (ssj_ops)	Average Power (Watts)
100%	865,618	258
90%	786,688	242
80%	698,051	224
70%	607,826	204
60%	521,391	185
50%	436,757	170
40%	345,919	157
30%	262,071	146
20%	176,061	135
10%	86,784	121
0%	0	80
Overall Sum	4,787,166	1,922
Σssj_ops/Σpower =		2,490

- Google data center
  - Mostly operates at 10%-50% load
  - At 100% load less than 1% of the time

- Designing processors to make power proportional to load

  Power is a limiting factor

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Pitfall: MIPS as a Performance Metric

- MIPS: Millions of Instructions Per Second
  - At 100% load : 258W
  - At 50% load : 170W (66%)
  - At 10% load : 121W (47%)
- Doesn't account for
  - Different ISAs between computers
  - Differences in complexity between instructions
- CPI varies between programs on a given CPU

$$\text{MIPS} = \frac{(\text{Instruction count})}{(\text{Execution time}) \times 10^6}$$

$$= \frac{(\text{Instruction count})}{\frac{(\text{Instruction count}) \times \text{CPI}}{\text{Clock rate}} \times 10^6}$$

$$= \frac{\text{Clock rate}}{\text{CPI} \times 10^6}$$

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Pitfall: MIPS as a Performance Metric

Example

Given the IC, Clock rate, and CPI of Computer A and B as below,

Measurement	Computer A	Computer B
Instruction count	10 billion	8 billion
Clock rate	4 GHz	4GHz
CPI	1.0	1.1

Which Computer has higher MIPS rating?

Which Computer is faster?

A.

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

RISC vs. CISC

RISC (Reduced Instruction Set Computing) architecture

- Example : MIPS architecture, ARM, PowerPC, etc.
- MIPS: **M**icroprocessor without **I**nterlocked **P**ipeline **S**tages, refers to a RISC architecture in the early 1980s
- Advantages: Simplified and Small Set of Instructions
  - Faster Execution
  - Power-efficient
  - Scalability: Easier implementation of features like parallelism
- Disadvantages: Simplified and Small Set of Instructions
  - More Instructions for Complex Tasks: increasing program size

CISC (Complex Instruction Set Computing) architecture

- Example : x86, x86-64 (Intel, AMD)
- Minimize the number of instructions per program by providing large and complex set of instructions
- Advantages
  - Reduced Program Size
  - Compatibility with older instructions
- Disadvantages
  - Slower Execution and Higher Power Consumption

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential



Thank you

Bumsoo Kim

School of Computer Science & Engineering

E-mail : bumsoo@cau.ac.kr

Office : 310-430